AI-200 TRAINING | CASE STUDY

ShopSphere

Designing a Scalable E-Commerce Application on Azure

Architecture, workflows, messaging, security, and observability

Purpose of this case study

Connect the Azure services learned in AI-200 to one realistic business scenario. Follow a customer journey from product browsing through order processing, then explore how the application is secured and monitored.

The business in one minute

ShopSphere sells electronics, clothing, and home appliances. Traffic rises sharply during flash sales and festive campaigns. The company needs a responsive website, dependable order processing, fast product access, secure configuration, and a clear way to diagnose failures.

CUSTOMER EXPERIENCE Fast browsing and cart updates	**RELIABLE ORDERS** Queue downstream work safely
SECURITY Keep secrets out of source code	**OPERATIONS** Trace requests and investigate logs

Designed for instructor-led AI-200 refresher discussion

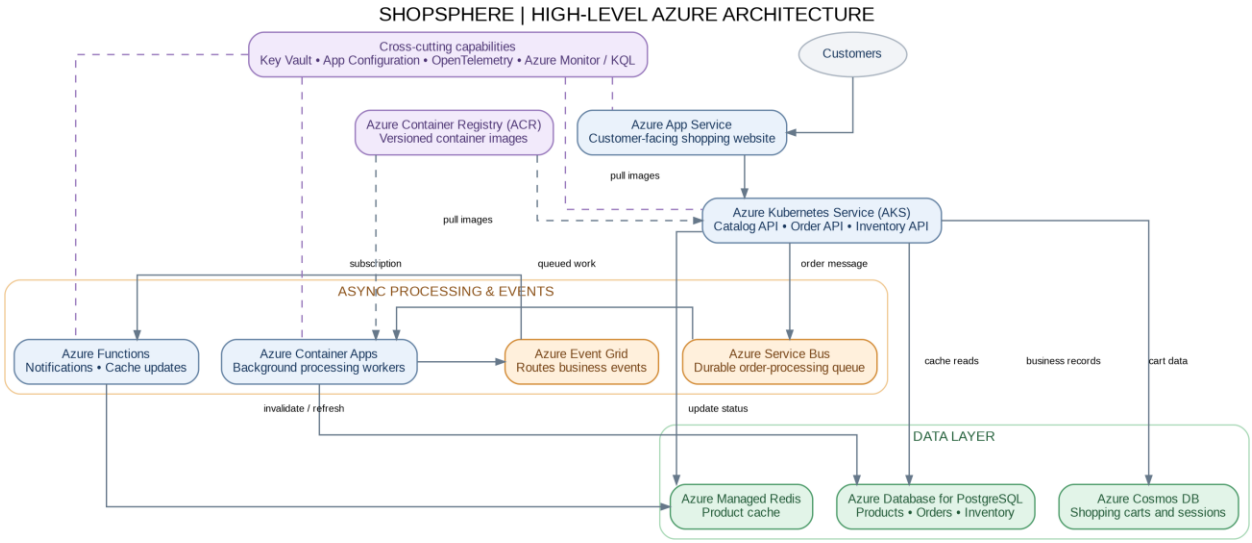
1. Business scenario and design goals

ShopSphere's original application handles ordinary shopping traffic but struggles during promotions. The team wants to modernize the platform without treating every operation as the same type of workload. Interactive web requests, order-processing work, event notifications, data persistence, and operational telemetry have different requirements.

Business pain points

PAIN POINT	BUSINESS IMPACT
Traffic spikes	The website slows down when many customers browse and check out at the same time.
Repeated database reads	Popular product details are fetched repeatedly, increasing latency and database load.
Coupled order processing	Invoice generation and notifications can delay the checkout experience if run synchronously.
Changing data needs	Cart/session data and relational order records have different data-access patterns.
Configuration and secrets	Credentials and environment-specific settings are difficult to manage consistently.
Limited troubleshooting	The team cannot easily follow a request across APIs, databases, and background workers.

High-level architecture



The architecture is conceptual. It shows service responsibilities and the main interaction patterns rather than every network, identity, or deployment detail.

2. Azure service roles at a glance

Each service has a defined responsibility. The architecture works because the team chooses the right service for each requirement and makes the interactions explicit.

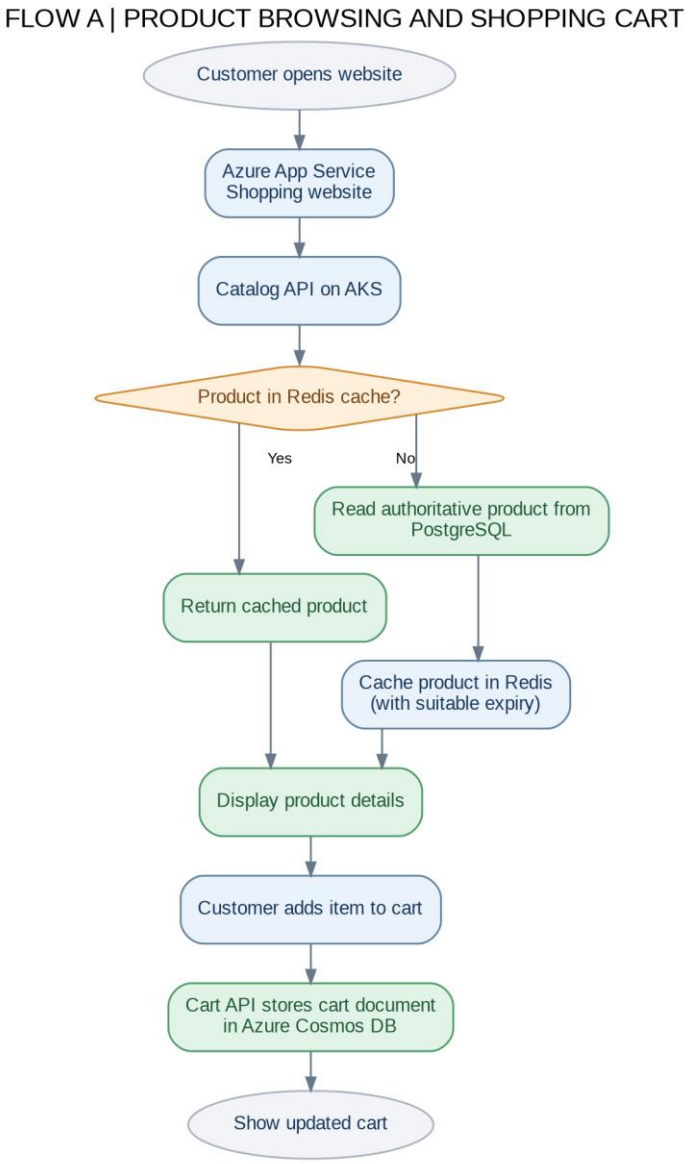
CAPABILITY	SERVICE	ROLE IN SHOPSPHERE
Application hosting	Azure App Service	Hosts the customer-facing shopping website.
Container images	Azure Container Registry (ACR)	Stores versioned images used by containerized workloads; it does not run containers.
Container orchestration	Azure Kubernetes Service (AKS)	Runs core APIs such as catalog, order, and inventory services where Kubernetes control is useful.
Containerized background work	Azure Container Apps	Runs background workers without requiring the team to manage a Kubernetes cluster for those workers.
Document data	Azure Cosmos DB	Stores shopping-cart documents and customer session data.
Relational data	Azure Database for PostgreSQL	Stores structured products, orders, inventory, and relational business records.
Caching	Azure Managed Redis	Caches popular product data to reduce repeated database reads and improve response time.
Reliable messaging	Azure Service Bus	Queues business work, such as order processing, so it can be handled asynchronously.
Event distribution	Azure Event Grid	Routes events, such as ProductUpdated or OrderConfirmed, to subscribed components.
Event-driven code	Azure Functions	Runs tasks triggered by messages, events, timers, or HTTP requests; examples include notifications and cache updates.
Secrets	Azure Key Vault	Protects credentials, API keys, certificates, and other sensitive values.
Application settings	Azure App Configuration	Centralizes non-secret settings and feature flags, such as enabling a promotion banner.
Instrumentation	OpenTelemetry	Collects standardized traces, metrics, and logs from instrumented application components.
Log analysis	Kusto Query Language (KQL)	Queries telemetry in Azure Monitor Logs to investigate failures and performance.

Design principle

Not every application needs both AKS and Container Apps. In this teaching scenario, AKS illustrates Kubernetes-based orchestration for core APIs, while Container Apps illustrates managed execution for background workers. A real design should justify the extra platform complexity.

3. Flow A - Product browsing and shopping cart

A customer searches for a laptop, opens its product details, and adds it to the shopping cart. The catalog API checks the cache before reading the authoritative product record.



What happens in this flow?

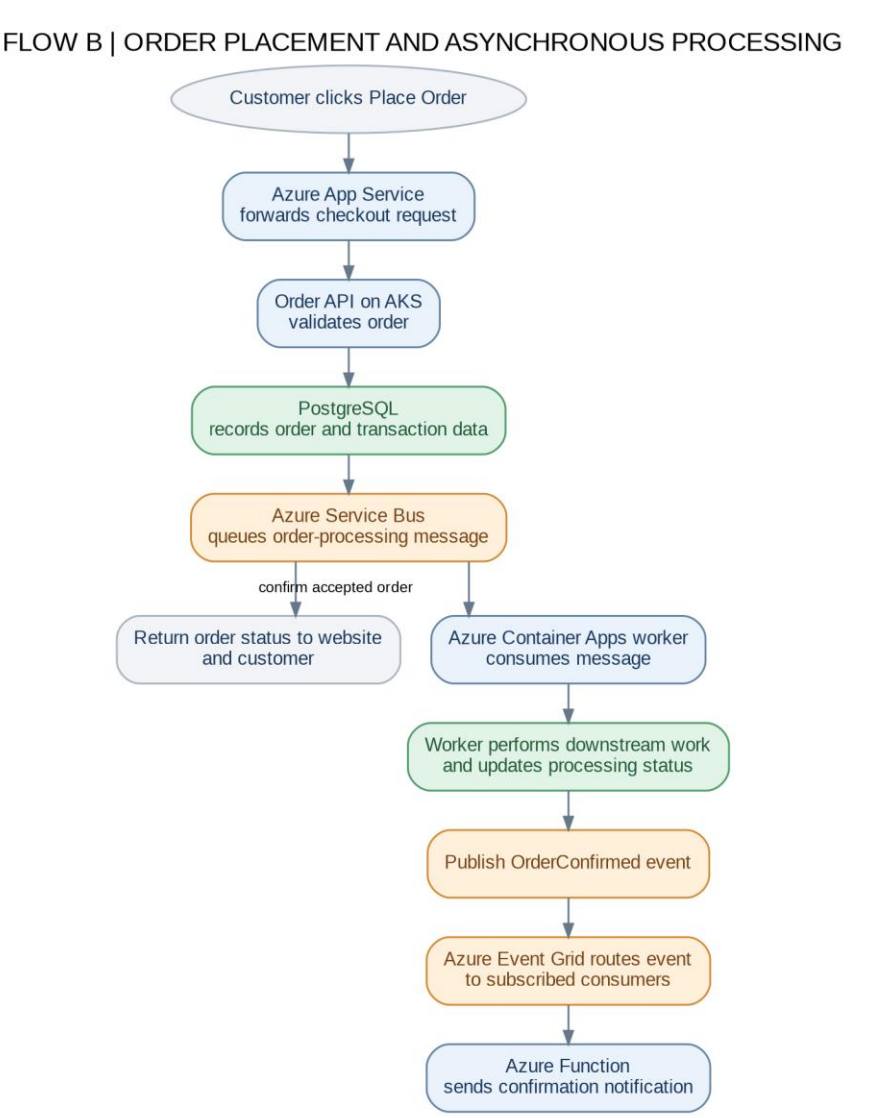
1. App Service delivers the website and forwards product requests to the catalog API hosted on AKS.
2. The catalog API checks Azure Managed Redis. If the product is cached and still valid, it returns the cached value.
3. On a cache miss, the API reads product data from PostgreSQL, stores a copy in Redis with a suitable expiration, and returns it.
4. When the customer updates the cart, the cart API stores the cart document in Cosmos DB.

Trainer emphasis

Redis is a performance cache, not the source of truth. Cosmos DB stores cart/session documents; PostgreSQL stores relational product and order data. Define expiry and invalidation rules to avoid stale prices.

4. Flow B - Order placement and asynchronous processing

The customer clicks Place Order. The application validates and records the order, then puts downstream work on a queue so that invoice generation, fulfilment preparation, and notifications can proceed independently.



Why use Service Bus?

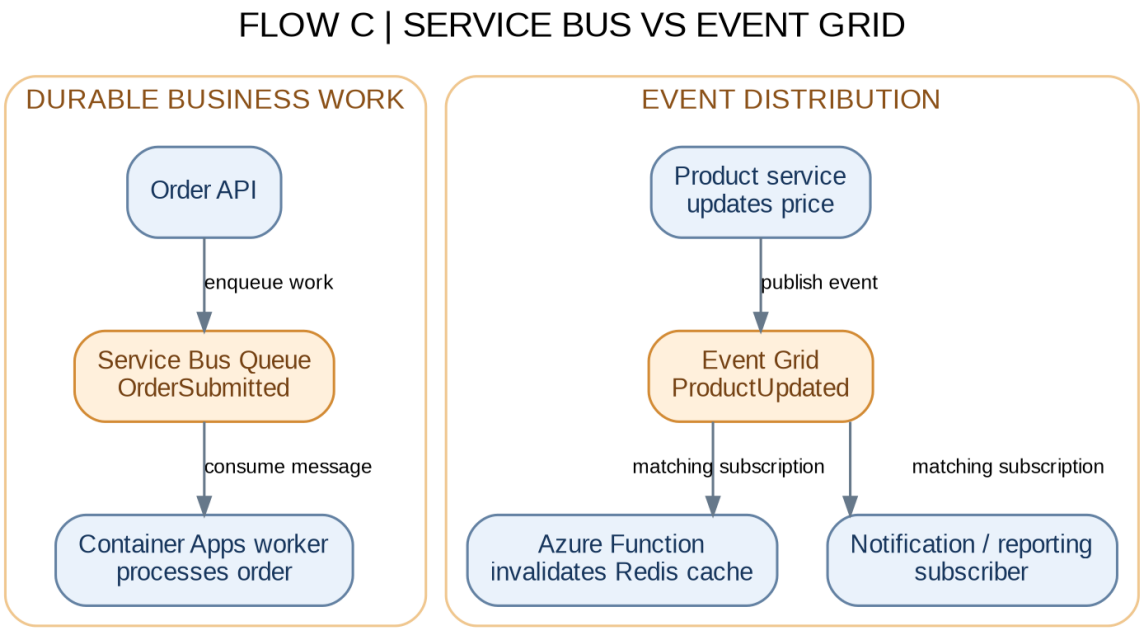
- It buffers work when order arrivals temporarily exceed the capacity of background workers.
- Workers can scale independently of the customer-facing web tier.
- Retries and dead-letter queues help operators handle messages that cannot be processed successfully.
- The website can return an order status without waiting for every downstream activity to finish.

Production design consideration

Saving an order in PostgreSQL and publishing a Service Bus message are separate operations. A production solution should handle partial failure, retries, and duplicate delivery. An outbox pattern plus idempotent message handlers can help avoid lost work and duplicate side effects.

5. Flow C - Service Bus versus Event Grid

Both services support asynchronous communication, but they answer different questions. Service Bus is used when an application needs work to be processed reliably. Event Grid is used when an event should be distributed to interested subscribers.



COMPARE	SERVICE BUS	EVENT GRID
Primary purpose	Reliable business messaging and work distribution.	Event routing and publish-subscribe distribution.
ShopSphere example	OrderSubmitted message goes to a worker that processes the order.	ProductUpdated event is routed to a cache invalidation function and other subscribers.
Mental model	“This work must be done.”	“This happened; interested systems may react.”
Typical consumer	A worker that receives and completes a message.	One or more subscribers selected by event subscriptions.

Example: a product price changes

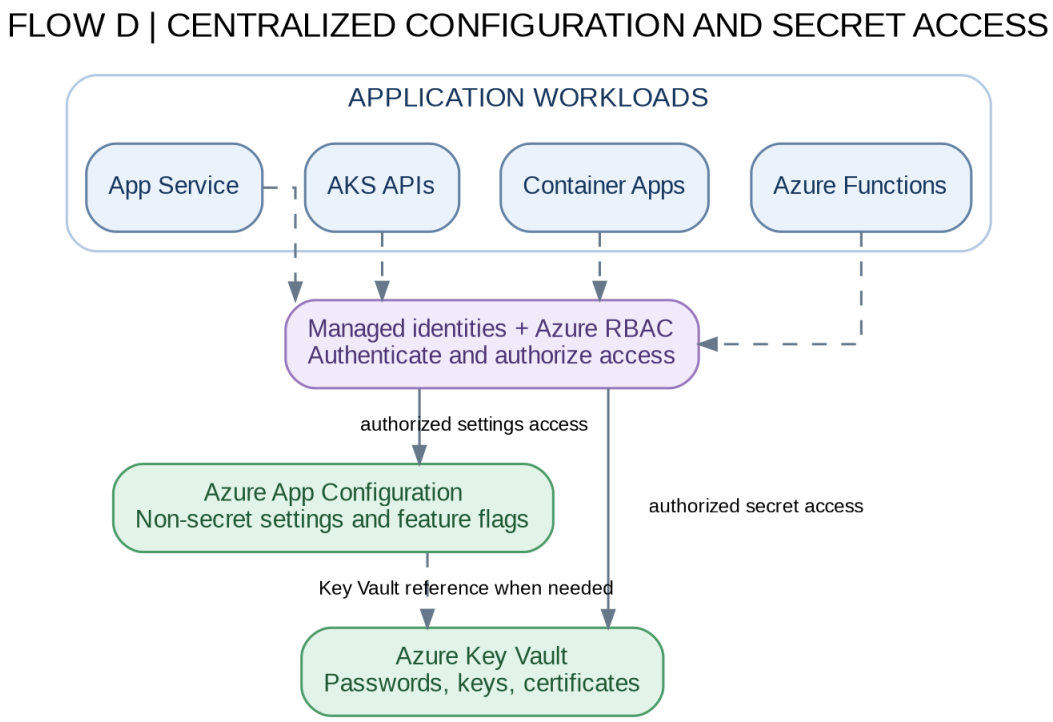
1. The product service saves the new price in PostgreSQL.
2. The service publishes a ProductUpdated event.
3. Event Grid routes the event to a subscribed Azure Function.
4. The function invalidates or refreshes the affected Redis entry; the next request then retrieves the current product value.

Remember

Azure Functions is the code that performs an action. Service Bus and Event Grid are communication services with different delivery patterns. A Function can be triggered by either, depending on the workflow.

6. Flow D - Centralized configuration and secrets

ShopSphere has multiple workloads: App Service, APIs on AKS, Container Apps workers, and Azure Functions. They need consistent settings and secure access to sensitive values without embedding passwords or API keys in source code or container images.



Azure Key Vault

- Stores sensitive values such as database credentials, payment-provider API keys, and certificates.
- Applications authenticate using managed identities where supported, and access is granted with least privilege.
- Avoid hardcoding secrets in source code, configuration files committed to source control, or container images.

Azure App Configuration

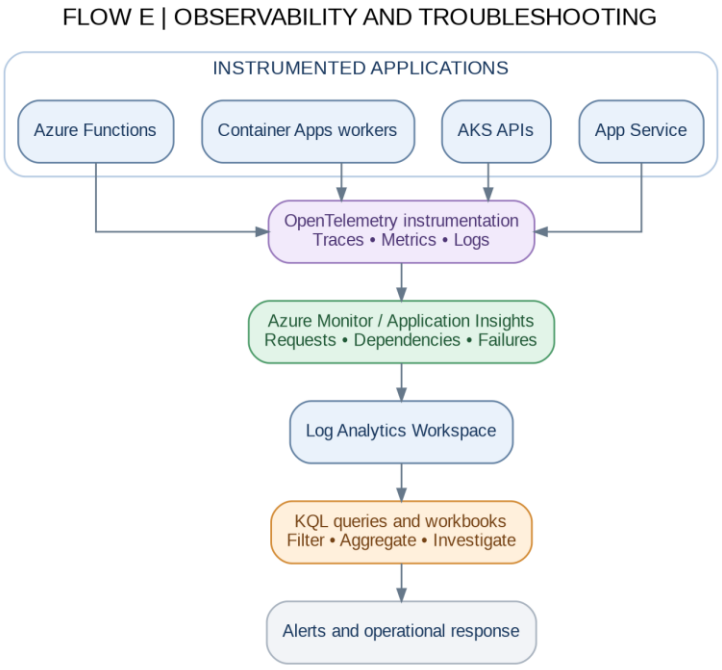
- Stores non-secret settings and feature flags centrally, such as the page size, promotion banner, or controlled rollout of a new checkout experience.
- Supports environment-aware configuration and runtime changes where the application is designed to refresh settings.
- Can use Key Vault references for settings whose values are stored securely in Key Vault.

Simple distinction

App Configuration manages application settings. Key Vault protects secrets. Managed identity and Azure RBAC help control which workload can access each resource.

7. Flow E - OpenTelemetry and KQL for troubleshooting

During a festive sale, customers report that checkout is slow. The engineering team needs to determine whether the delay comes from the order API, a database dependency, or background processing.



How the team investigates

- OpenTelemetry instruments application components and captures distributed traces, metrics, and logs.
- Azure Monitor and Application Insights help investigate request duration, failures, and dependency timings.
- Log Analytics stores queryable log data. KQL filters and summarizes telemetry to identify repeated failures or slow operations.

Example KQL - find failed or slow requests in the last hour

```
AppRequests
| where TimeGenerated > ago(1h)
| where Success == false or DurationMs > 2000
| project TimeGenerated, Name, DurationMs, ResultCode, Success
| order by TimeGenerated desc
```

This query assumes the workspace uses the Application Insights AppRequests table schema. The team can then open related traces and dependency records to understand where the request is spending time.

Trainer emphasis

OpenTelemetry is an instrumentation and telemetry approach; KQL is a query language. Together with Azure Monitor, they help teams move from “checkout is slow” to evidence about the affected operation or dependency.

8. Service responsibility quick reference

NEED	SERVICE
Website hosting	App Service
Container image storage	ACR
Core API orchestration	AKS
Background container workers	Container Apps
Shopping cart documents	Cosmos DB
Relational business records	PostgreSQL
Frequently accessed product cache	Managed Redis
Durable order-processing work	Service Bus
Business-event routing	Event Grid
Event-triggered application code	Azure Functions
Secrets and certificates	Key Vault
Non-secret settings and feature flags	App Configuration
Tracing, metrics, and logs instrumentation	OpenTelemetry
Querying operational telemetry	KQL

Discussion questions for learners

1. What happens when a product request is a Redis cache miss?
2. Why is Cosmos DB used for cart documents while PostgreSQL stores relational order data?
3. Why does checkout queue downstream work instead of waiting for invoice and notification processing to finish?
4. How would you decide whether an operation belongs in Service Bus or Event Grid?
5. When is an Azure Function sufficient, and when would a continuously running Container Apps worker be more suitable?
6. How do managed identity, Key Vault, and App Configuration work together?
7. How would OpenTelemetry traces and KQL help investigate a slow checkout operation?

Key design considerations

- Use each service for a clear reason; do not add complexity just because a service is available.
- Design around transient failures, retries, duplicate messages, and cache invalidation.
- Apply least-privilege identity and network access. Monitor queue depth, latency, error rates, and resource utilization.
- Use the right hosting model for each workload and review the operational cost of running multiple compute platforms.

Final takeaway

A modern Azure application is not just a collection of deployed resources. It combines purposeful service boundaries, reliable communication, secure configuration, appropriate data stores, and end-to-end observability.